Clase 5

siempre medico de guardia, y hay dos medicos, uno que tiene que estar, y entonces dicen che hay dos medicos, si, entonces me voy, y los dos se fueron. Como prevengo eso?

concurrencia con gente que esta tratando de hacer cosas y modifica los estados, tratar de evitar invariantes. Tengo que prevenir que los dos medicos hagan eso al mismo tiempo

el oro no puede ser negativo

si tengo 2 telefonos con mi cuenta, el intento de ingresar no se puede.

De orden global a correctitud

- consenso → orden total de operaciones
- Falta: correctitud logica de operaciones compuestas

> Problema: como ejecutar multiples operaciones concurrentes sin violar invariantes?

2 tratando de leer un registro y los dos leen al mismo tiempo y los dos intentan de cambiar. El invariante en esto: no te deberias perder ninguna operacion.

Invariante: es la regla que uno se tiene que asegurar, por mas de que pasen mil operaciones al mismo tiempo. Son como garantias, solo que hay que mantener, a pesar de todas las operaciones concurrentes y ante fallas.

Transaccion

> Definicion: unidad logica de trabajo que debe ejecutarse de fomra atomica y aislada

ACID:

▶️ Atomicidad: todo o nada
▶️ Consistencia: preserva invariantes

▶️ Aislamiento: ejecución como si fuera sola
▶️ Durabilidad: persistencia tras commit

solo es serializable cuando importa el orden, si un grafo no tiene un ciclo, no es serializable.

Anomalias

cuando tengo que implementar en la vida real transacciones compuestas como estas, que las quiero atomicas, y quiero asegurar un orden seguro secuencial. ES LENTO Y COMPLICADO DE MANTENER. Aveces quiero relajar alguna de las invariantes, para optimizar tiempos y recursos, entonces permito que no todo sea organizado. Empezar a relajar garantias, y empiezan a haber potenciales problemas que decidis aceptar

Dirty Read (leer un dato no commiteado)

un proceso escribio un dato T1 (transaccion atomica), T2 lee el valor no confirmado, T1 aborta

permito en: # de personas en el servidor, el directorio de zonas

Non-repeatable read

- T1 lee valor
- T2 lo modifica
- T1 vuelve a leer distinto valor

Phantom

▶️ T1 lee conjunto
▶️ T2 inserta nuevo registro
▶️ T1 vuelve a leer ⇒ aparece “fantasma”

Write skew

el problema de los médicos. 2 transacciones leen el mismo estado, y tratan de grabar, sin darse cuenta que estaban en conflicto los dos. Se pisan los valores.

- ambas transacciones leen mismo estado
- ambas escriben sin conflicto directo

sucede solo si rompen una garantia, una invariante

Niveles de aislamiento

SQL estandar

- read uncommitted
- read committed
- repeatable read: cuando empiezo a leer, la bd se tiene que guardar en una copia el estado, para poder tener repeatable, porque si alguien los modifica despues de que empece, no voy a leer lo mismo
- serializable

Problema:

Commit en multiples nodos debe ser atomico

2PC

Fase 1: Prepare

- coordinador pregunta: puedes commitear

Fase 2: Commit

- si todos dicen si → commit
- si alguno dice no → abort

aca tiene que haber uninamidad, en paxos es por mayoria, si uno dice que no, tenes que abortar todos.

si el coordinador se cae, despues de haber mandado el prepare, pero no recibio los okay, el resto quedan bloqueados, porque no saben que paso. Se te cuelga todo porque los otros servidores no pueden aceptar transacciones.

3PC

- agrega fase intermedia (pre-commit)
- reduce bloqueo

Limitacion:

- requiere sincronía parcial

Commit distribuido

compensar la saga con compensasiones, por ejemplo que ya le saque la plata y despues en otro paso algo fallo, pero la transaccion esa se commiteo (no se puede hacer rollback), entonces hago otra transaccion para devolverle la plata (estoy compensando)

> Idea central: serializabilidad es el estandar de correccion, pero requiere coordinacion fuerte.